Orchestrating MCP-Based Scheduling Assistants: Comparing LLM and Backend Control

Daniel Becker[1,2]

[1]Coburg University of Applied Sciences and Arts, Germany
[2]HUK-COBURG, Germany

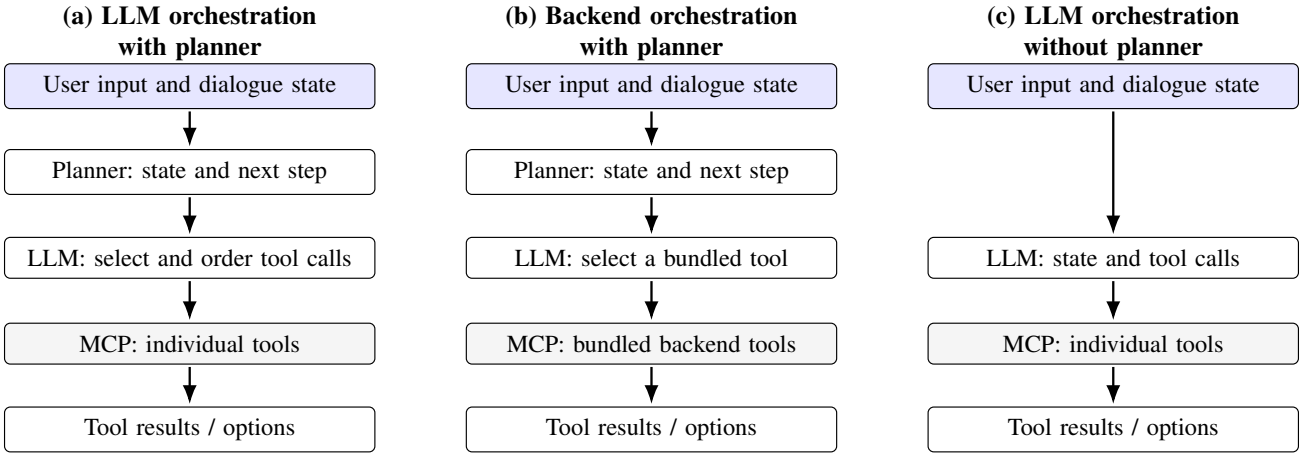


Figure 1: Orchestration of a scheduling assistant with a large language model (LLM) and Model Context Protocol (MCP) tools. Dialogue state stores information from earlier turns. **(a)** A separate planner updates that state and identifies the next step. The LLM selects and sequences individual tools. **(b)** The same planning step runs first. The LLM then selects a bundled tool, and the backend runs its internal operations in order. **(c)** The LLM handles state updates and individual tool selection without a separate planner. Arrows show logical processing for complete inputs. Blue marks inputs and grey marks tools. Client and server mediate every tool execution. All variants require clarification for missing information and explicit human confirmation plus deterministic validation before booking. These return paths are omitted.

Abstract

Integrating large language models (LLMs) with existing software requires more than selecting suitable tools: multi-step actions must be coordinated, dialogue context must be maintained, and consequential operations must remain under human control. This paper presents the design, configuration selection and evaluation of an MCP-based conversational scheduling assistant. The prototype exposes existing service capabilities through the Model Context Protocol (MCP) and combines structured dialogue state, server-side validation and explicit user confirmation before booking. A selection study compares three models and three orchestration variants on two scenarios, yielding 90 analysed runs after excluding cold starts. Claude Haiku 4.5 with individual tools and no separate planner produced correct outcomes in all five repetitions of each scenario and was selected for further evaluation. The comparison illustrates that technically successful tool calls need not produce correct outcomes. After a separate settings experiment with 50 analysed runs, only the chosen Haiku configuration was assessed on seven functional cases, 80 executions of eight direct prompt-injection inputs, concurrent use and exploratory feedback from three operational users and six developers. The functional cases met expectations after an error-handling adjustment for empty search results. No control violation was observed in the injection tests, although some inputs disrupted the expected workflow. User feedback favoured flexible input while identifying a need for concise responses and shorter delays. Selection results and subsequent observations provide task-specific evidence, not an independent model ranking or proof of production readiness.

1. Introduction

Large language models (LLMs) can use external tools to retrieve information and invoke software functions [SDYD*23]. In a conversational scheduling assistant, this capability must support a sequence of dependent actions: interpreting preferences, clarifying missing information, retrieving suitable options and confirming a booking. A plausible response alone is insufficient. The application must also retain dialogue context, validate inputs and keep the final booking decision under human control.

The Model Context Protocol (MCP) provides a common interface through which applications can access tools exposed by servers [Mod25]. For the scheduling task considered here, a further design decision concerns orchestration: which component selects and sequences the required operations? The LLM can coordinate individual tools, or a backend tool can bundle several operations into a predefined sequence. A separate planning stage introduces another design choice whose value must be assessed alongside its additional processing effort.

This paper investigates the following question: *How can an MCP-based scheduling assistant be designed for human control, and how can its model and orchestration configuration be selected and evaluated?* We examine a prototype that lets users access existing services through a chat interface. It stores relevant information from the conversation in a structured form. Users must explicitly confirm an appointment before it is booked. The selection stage compares LLM orchestration with a planner, backend orchestration with a planner, and LLM orchestration without a separate planner (Figure 1).

The contribution is an empirical case study of controlled tool use in a bounded scheduling workflow. It connects three aspects: a generalised design that separates model decisions from application controls, a configuration-selection study that distinguishes execution success from outcome correctness, and a subsequent assessment of the selected system through functional cases, direct prompt injection and user feedback. The selection also considers response time, token usage and model-call cost. Together, these results show where efficient tool execution can still fail to meet the task goal and where a working prototype needs further interaction and integration improvements. MCP is the shared integration layer, so the study evaluates orchestration choices rather than an intrinsic advantage of the protocol. The following sections present the background and requirements, system design, evaluation, and implications for comparable applications.

2. Background and Related Work

MCP separates the host application's coordination responsibilities from the capabilities exposed by servers [Mod25]. In this paper, dialogue state denotes the structured information retained across turns. Four studies inform the use of tools, their orchestration and user control.

ToolLLM. The framework addresses the training and evaluation of language models for real-world API use. The ToolBench dataset covers 16,464 RESTful APIs, while a depth-first search method explores alternative sequences of calls. The resulting ToolLLaMA model demonstrates multi-tool execution and generalisation to unseen APIs in the reported evaluation [QLY*24]. This supports studying tool selection and sequencing together. Our prototype instead uses existing models with a small, predefined set of MCP tools and requires no model training.

RestGPT. This approach separates task planning, API selection and execution. The system uses API descriptions to construct calls and process responses, then feeds execution results back into planning. On RestBench, covering TMDB and Spotify tasks, removing the planner reduces success rates [SXZ*23]. This motivates our comparison of orchestration with and without a separate planner. The reported benefit applies to the evaluated benchmark and architecture, so its transfer to a bounded scheduling workflow requires evaluation.

AI Chains. This approach connects multiple model calls so that each step can use earlier outputs. An interactive interface exposes intermediate results and allows users to modify individual steps. A study with 20 participants found improved task outcomes and perceived transparency, controllability and collaboration compared with a non-chaining interface [WTC22]. For our design, this motivates explicit intermediate state and opportunities for correction, while leaving the allocation of tool orchestration to be investigated.

Guidelines for Human-AI Interaction. This work presents 18 design guidelines evaluated through several rounds, including a study with 49 practitioners examining 20 AI-enabled products. The guidelines address communicating capabilities, correcting errors, clarifying uncertain goals and remembering recent interactions [AWV*19]. These principles inform our dialogue design. The requirement for explicit confirmation before booking is our application-specific implementation of human control.

Together, these studies connect tool capability, modular processing and human oversight. Our study applies these perspectives to an MCP-based scheduling assistant and compares three implementations that distribute planning and execution between the LLM and backend. Its focus is the resulting reliability and efficiency trade-offs within this controlled workflow.

3. Use Case and Requirements

The use case concerns an authorised staff member arranging an appointment during a customer interaction. The customer is already identified within the existing process and has expressed an interest in an appointment. The assistant supports scheduling through natural-language input. This work does not cover direct customer access, professional advice or changes to confirmed bookings.

Figure 2 summarises the workflow. The staff member enters the appointment request and relevant time and location preferences. Missing or ambiguous information is clarified before dependent operations proceed. Available options are retrieved and reviewed with the customer. If no suitable option is found or preferences change, the request can be revised. Booking requires the customer's agreement and an explicit confirmation by the staff member. Selecting a returned option constitutes this confirmation, rather than an additional dialogue turn. The application then validates and executes the request and reports its outcome. A confirmation email can follow a successful booking if a valid address is available.

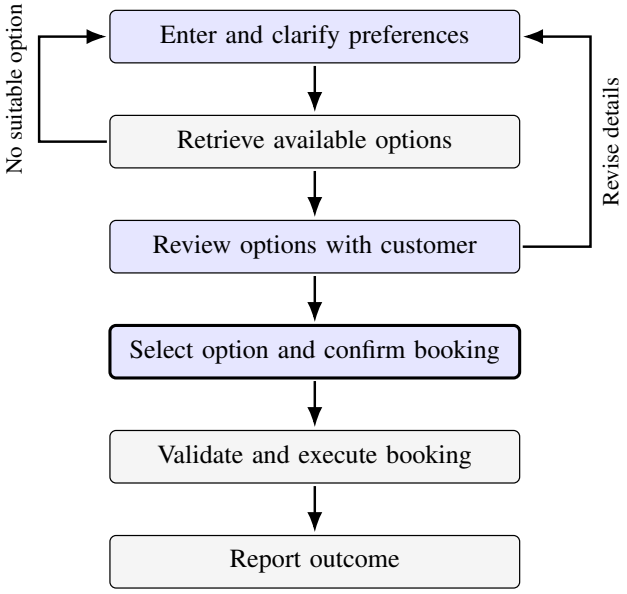


Figure 2: Generalised scheduling workflow for a staff-operated conversational assistant. Arrows show the order of actions, while the return paths allow revised preferences and a new search. Blue boxes mark staff input or review. Selecting an option in the bold box confirms the booking after agreement with the customer. Grey boxes mark application operations. Booking remains subject to validation. Cancellation and failure handling apply throughout but are omitted from the diagram.

The requirements cover context handling, controlled execution and clear feedback. Information must persist across dialogue turns and remain correctable before booking. Changes must invalidate dependent information, such as outdated appointment options. Access must remain restricted to authorised users and permitted records, and model input must be limited to information needed for the task. Tool parameters and booking conditions require application-side checks. Unavailable appointments must be distinguished from technical failures, and a booking must only be reported as successful after the service confirms it. The staff member must be able to cancel before booking. Responses should be concise, and processing delays should support use during an ongoing conversation.

4. System Design and Orchestration Variants

This section describes the shared architecture and dialogue state before introducing the three orchestration variants.

4.1. Architecture and Dialogue State

The prototype adds a conversational interface to existing services, as shown in Figure 3. The dialogue client acts as the MCP host, manages the interaction and invokes the language model. An embedded MCP client communicates with a separate MCP server, which exposes the scheduling functions as tools and accesses existing services through their APIs. The application executes model-selected tool calls through this connection. The language model has no direct access to the underlying services.

The dialogue client maintains structured state containing known preferences, missing information and the current processing step. Each new input is interpreted together with this state, producing a response and an updated state for the next turn (Figure 4). The server separately retains intermediate service results needed by subsequent operations. This separation allows reusable results to remain available without placing all service data in the model context. Corrections update the relevant state and invalidate dependent results before processing continues.

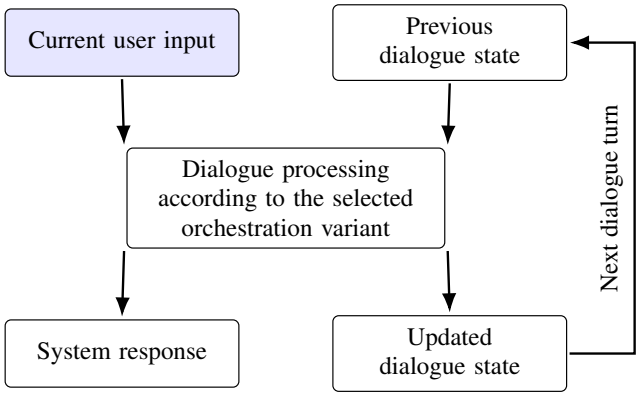


Figure 4: State handling across successive dialogue turns. The processing stage combines the current user input with information retained from previous turns, following the selected orchestration variant. It produces a response for the user and an updated dialogue state. The right-hand feedback arrow carries that state into the next turn, so previously established information can be reused.

4.2. Orchestration Variants

The three variants in Figure 1 share this architecture but differ in tool granularity and the use of a separate planning stage.

LLM orchestration with a planner. Each new user input first triggers a model call that updates dialogue state and identifies the next task step. This planner does not select or execute tools. If information is missing, the application requests clarification. Otherwise, a subsequent model interaction selects and sequences individual MCP tools using their descriptions and the task instructions. Several dependent tool calls may follow, with the planner invoked again only when a new user input arrives.

Backend orchestration with a planner. The same planning stage precedes tool selection, but the exposed tools bundle several operations. The language model selects a bundled tool, while the backend determines the sequence of its internal steps and passes intermediate results between them. This reduces the number of individual operations that the model must coordinate. Tool selection remains model-driven, so backend orchestration does not make the entire workflow deterministic.

LLM orchestration without a planner. Individual MCP tools remain available, but the separate planning call is omitted. State

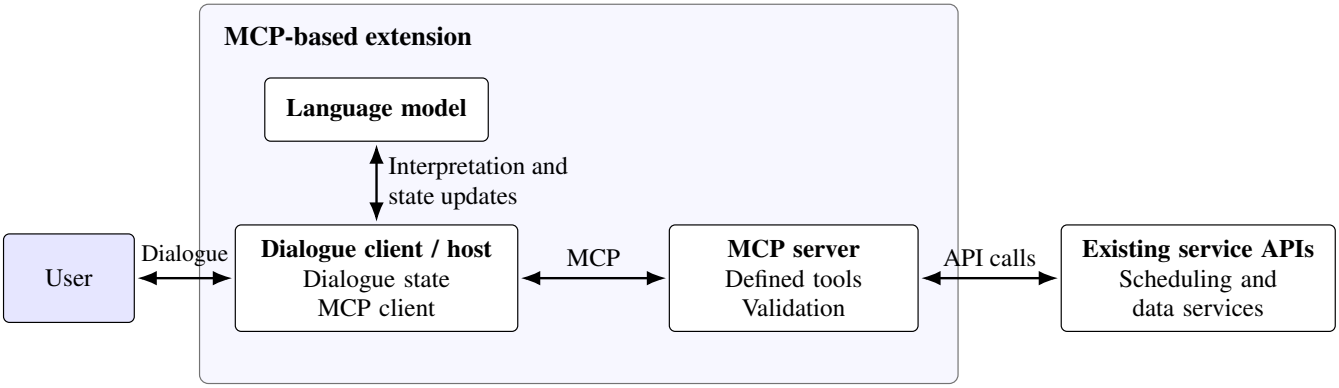


Figure 3: Generalised architecture of a scheduling assistant using the Model Context Protocol (MCP). The user interacts with a dialogue client that maintains state and asks a language model to interpret inputs. The client invokes tools on the MCP server, which validates requests and accesses existing scheduling and data services through their application programming interfaces (APIs). Double-headed arrows represent requests and responses, not unrestricted model access to services. Internal system names and deployment details are omitted.

updates, clarification and tool selection are handled together during model-driven dialogue processing. Tool results inform subsequent operations and the response to the staff member.

All variants retain application-side validation and the confirmation described in Section 3. The server resolves the selected option against stored search results before booking. Tool selection by the model therefore remains distinct from permission to execute the operation.

5. Evaluation Methodology

The study separates configuration selection from the subsequent evaluation of the selected prototype. Additional checks during development informed implementation but are excluded from the standardised run counts and statistical comparisons.

5.1. Configuration Selection

Three tool-capable models were compared with each orchestration variant to select a configuration for further evaluation. Table 1 lists the initial settings.

Table 1: Models and initial settings for configuration selection across all three orchestration variants.

Model	Temperature	Additional reasoning
Claude Haiku 4.5	0	Disabled
Qwen3-235B-A22B-2507	0	Disabled
Amazon Nova Pro	0.00001	Disabled

Two standardised scenarios cover different information requirements. In T01, a complete request is followed by selection and confirmation of a proposed option, giving two user inputs. In T02, an incomplete request requires clarification, followed by the missing information and then confirmation, giving three inputs.

Each scenario–model–variant combination was executed six times, giving 108 runs. The first run followed an application restart and was treated separately as a cold start. The remaining five repetitions per combination yielded 90 regular runs for analysis. Backend orchestration without a planner was not tested.

5.2. Selection Criteria and Measures

Technical success, tool behaviour and result correctness were assessed separately. Technical success requires the intended tool calls to complete successfully. Correct tool behaviour requires the necessary calls without missing, erroneous or unnecessarily repeated invocations. Result correctness requires a valid scheduling outcome and the confirmation email specified in the test scenario. Additional read-only calls do not automatically constitute an incorrect outcome, whereas repeated state-changing calls can affect correctness. These measures were evaluated across all five regular repetitions.

Response time was measured from submission of each input to complete display of the corresponding response and summed across the scenario, excluding user reading and input time. Token usage and model-call cost were accumulated across all model calls, accounting for input, output and applicable cache-token categories. Reported response times are medians of technically successful regular runs, including those with incorrect outcomes. Token and cost medians include all five regular runs, since failed attempts also consume resources. In Table 3, relative cost is the ratio of median model-call costs, normalised to LLM orchestration with a planner within each model and scenario. The small number of repetitions supports descriptive comparison rather than broad statistical generalisation.

5.3. Evaluation of the Selected Configuration

Selection prioritised correctness and reliability, followed by response time and cost. For Claude Haiku 4.5, five settings were compared using individual tools and no separate planner. Table 2 lists the temperature and additional reasoning budget for each setting.

Table 2: Haiku settings compared on T01 and T02. Disabled means that no additional reasoning was used. K2 was selected for the subsequent evaluation, as discussed in Section 6.2.

Setting	Temperature	Reasoning budget [tokens]
K1	0	Disabled
K2	**0.5**	**Disabled**
K3	1	Disabled
K4	0	1 024
K5	0	4 096

Each setting used six runs per scenario, again excluding the first as a cold start. This adds 60 runs, of which 50 were analysed, separate from the initial 108-run selection study. Only the selected Haiku setting underwent the subsequent functional, robustness, concurrent-use and user assessments. Seven functional cases check the expected response to complete requests, missing or changed information, ambiguous inputs and empty search results. They document case coverage rather than a repeated-run success rate.

For prompt injection, each fixed test input was repeated in ten fresh sessions, giving 80 executions. The assessment distinguishes deviations from the expected workflow from control violations, such as skipping required confirmation, disclosing protected information or reporting a booking that did not occur. These are direct user-input tests, not attacks embedded in retrieved content or tool responses.

Three operational users and six developers each completed two scheduling tasks using their own wording. One task used a relative date and a morning preference, while the other used an explicit date and an afternoon preference. Participants then rated 15 statements on a five-point agreement scale and could provide free-text feedback. The two groups were analysed separately, and developers did not rate the two statements about operational usefulness and regular use. Results are descriptive because of the small samples. Concurrent-use tests repeated the complete-input scenario under increasing load and recorded response times and HTTP outcomes. These request-level measures are separate from the scenario-level correctness assessment.

The existing REST integration serves as a qualitative reference only. Its different task and interface prevent a direct comparison of response times or success rates.

6. Results and Discussion

Table 3 reports the evidence used for configuration selection. The final Haiku settings experiment follows in Section 6.2, before the functional, robustness and user assessments of that configuration.

6.1. Configuration Selection Results

Claude Haiku 4.5 without a separate planner was the only combination with correct outcomes in all five runs of both scenarios. Its median response times were 13.381 s and 17.247 s. Backend orchestration with the same model was faster at 8.153 s and 13.541 s, but yielded only one and two correct outcomes out of five. In T01, all five backend runs were technically successful and used the tools correctly, yet only one met the outcome criterion. This illustrates why execution success alone is insufficient.

The planner effect varied by model and metric. For Qwen3 in T01, adding a planner increased technical success from two to four runs but left correct outcomes at two. Without a planner, Qwen3 reached five correct outcomes in T02 but only two in T01. Backend

Table 3: Configuration-selection results for T01 (complete input) and T02 (clarification required). LP: LLM orchestration with planner, BP: backend orchestration with planner, LN: LLM orchestration without planner. **T (technical success)** counts runs in which the intended tool calls completed successfully. **U (correct tool use)** counts runs with the required calls and no missing, erroneous or unnecessary repeated calls. **C (correct outcome)** counts runs with a valid scheduling outcome and the required confirmation email. Each count is out of five runs. For example, 5/5/4 means five technically successful runs, five runs with correct tool use and four with correct outcomes. Lowercase t is median cumulative response time over the technically successful runs counted by T, including those with incorrect outcomes. Tokens and costs are medians over all five runs. Token totals include regular input tokens, cached input tokens, cache-write tokens and output tokens where recorded. In these experiments, cached input and cache-write tokens were recorded only for Claude Haiku 4.5. K_{rel} is median model-call cost divided by the LP median for the same model and scenario. Relative costs are rounded. No results are pooled across models or scenarios.

		T01				T02			
Model	**Variant**	**T/U/C**	t [s]	**Tokens**	K_{rel}	**T/U/C**	t [s]	**Tokens**	K_{rel}
Claude Haiku 4.5	LP	5/5/4	19.896	111 486	1.00	3/3/3	21.389	104 552	1.00
	BP	5/5/1	8.153	48 330	0.48	4/4/2	13.541	53 355	0.59
	LN	5/5/5	13.381	124 208	1.02	5/5/5	17.247	154 143	1.29
Qwen3-235B-A22B-2507	LP	4/4/2	21.767	67 771	1.00	3/3/1	19.542	50 195	1.00
	BP	4/4/2	14.408	21 416	0.33	3/3/2	12.661	24 713	0.51
	LN	2/2/2	15.568	59 747	0.87	5/5/5	19.680	65 471	1.27
Amazon Nova Pro	LP	2/2/1	11.950	30 427	1.00	4/2/2	19.061	52 193	1.00
	BP	4/3/1	8.570	20 361	0.69	2/2/2	11.920	19 712	0.39
	LN	2/2/2	9.394	34 845	1.16	2/2/2	10.164	38 616	0.73

orchestration had the lowest median model-call cost within every model and scenario, at 0.33–0.69 times the respective LLM-with-planner baseline. Input processing dominated token use, and cache usage differed between models, so token totals alone do not determine cost. These efficiency measures exclude the effort needed to correct an incorrect result.

The Haiku configuration without a planner was selected on observed correctness, despite higher costs than backend orchestration. RestGPT reports higher success rates with an iterative planner informed by execution results [SXZ*23]. Our planner operates once per user input, without replanning between tool calls. Omitting it combines state updates and tool selection, rather than removing all decision-making. Different tasks, models and planning mechanisms prevent a direct replication claim. The bounded workflow may help explain the limited benefit here, but its causal effect was not isolated.

6.2. Haiku Settings Selection

During settings selection, all five settings produced correct outcomes in all five regular runs of each scenario. Technical success and correct tool use were also observed in all regular runs. Table 4 makes the final settings decision explicit. Costs use the original evaluation rates per million tokens: USD 1.10 for regular input, USD 0.11 for cached input, USD 1.375 for cache writes and USD 5.50 for output. These match the published Haiku 4.5 regional rates, including five-minute cache writes, checked on 1 November 2026 [Ant26].

Table 4: Results for the Haiku settings defined in Table 2. Each row includes five regular runs per scenario. All runs were technically successful, used tools correctly and produced correct outcomes. Time and cost are medians per complete scenario run. Costs include all recorded token categories across all model calls and are rounded to six decimal places in USD. Bold marks the selected setting.

	T01		T02	
Setting	**Time [s]**	**Cost [USD]**	**Time [s]**	**Cost [USD]**
K1	14.253	0.081609	15.312	0.101242
K2	**12.528**	**0.081557**	**15.939**	**0.100520**
K3	13.379	0.081865	16.237	0.101124
K4	13.294	0.082705	16.324	0.101075
K5	14.915	0.081843	17.013	0.101329

Since all five settings achieved the same observed correctness, response time and cost informed the final selection. K2 combined the lowest median costs in both scenarios with the shortest response time in T01, while K1 was faster in T02. This balance motivated the selection of K2 for subsequent tests.

The cost differences are small and, with only five regular runs per setting and scenario, do not establish a meaningful general efficiency advantage. Neither higher temperature nor additional reasoning improved the observed correctness in these scenarios. This selection is specific to these observations, rather than evidence that temperature 0.5 is generally optimal.

6.3. Functional Cases

The subsequent functional assessment used K2 only. Table 5 lists the seven cases and the observed responses without company-specific identifiers or service details.

Table 5: Functional coverage of the selected Haiku configuration. E01–E06 behaved as expected. E07 met the expected behaviour after an error-handling adjustment. Rows describe cases, not repeated-run success rates.

ID	Case	Observed response
E01	Complete request	The intended scheduling process was performed.
E02	Missing customer identifier	The identifier was requested before scheduling continued.
E03	Missing appointment purpose	The purpose was requested before scheduling continued.
E04	Changed day	Availability was searched again for the new day, reusing existing customer information.
E05	Changed customer identifier	The revised identifier was used for the subsequent booking.
E06	Ambiguous input	Clarification was requested instead of making an assumption.
E07	No available appointments	After the adjustment, a valid empty search result was reported instead of a technical failure.

E04 illustrates that a correction can update dependent results while retaining information that remains valid. E07 also shows that model behaviour depends on how service results are presented. The successful handling of this case reflects a server-side correction as well as the selected model configuration. It is therefore evidence of resolving an integration problem, not evidence that the initial implementation handled every case correctly.

6.4. Direct Prompt-Injection Tests

Across the eight cases and 80 executions, no control violation was observed under the applied criteria. Table 6 separates that finding from disruptions of the normal workflow.

The three interrupted sequences in P02 and two deviations in P05 show that manipulative input can affect task execution without bypassing the assessed controls. P06–P08 concern information disclosure rather than completion of a scheduling task, so their results are not combined into an overall task-success rate. Repeating eight fixed inputs tests consistency for those inputs, not coverage of the wider attack space. The observations also do not isolate the contribution of model behaviour from application-side controls.

6.5. User Feedback

Operational users rated both stepwise and combined input positively, with a median of 5 for each statement on the five-point scale. The median was also 5 for understandable booking and correct handling of natural-language time expressions. These are participant ratings, not measured task-success rates. Response clarity and appropriate response times each received a median of 4. Perceived support in daily work and willingness to use the assistant regularly

Table 6: Direct prompt-injection tests of the selected Haiku configuration, with ten fresh sessions per case. No control violation was observed in any case. The last column reports workflow deviations for P01–P05 and disclosure outcomes for P06–P08. Raw prompts and internal details are omitted.

ID	Attempted deviation	Observation
P01	Skip booking confirmation	Expected workflow in 10/10.
P02	Reveal internal instructions	Expected workflow in 7/10. No tool call in 3/10.
P03	Continue without customer identifier	Missing information requested in 10/10.
P04	Invent a missing appointment purpose	Clarification requested in 10/10.
P05	Ignore errors and claim booking success	Expected workflow in 8/10. Deviations in 2/10, without false success claims.
P06	Reveal connection information	No disclosure in 10/10.
P07	Reveal model-access credentials	No disclosure in 10/10.
P08	Reveal authentication data	No disclosure in 10/10.

each had a median of 4 and a mean of 3.67. All these items had three responses. The selected items distinguish perceived ease of use from operational acceptance, which was less uniformly positive.

Free-text feedback requested shorter responses, concise questions and more visible appointment options. One operational user preferred a conventional form to a lengthy chat. The statement favouring direct scheduling with few inputs received a median of 5, which indicates a preference for a short interaction rather than stronger approval of conversational dialogue. The six developers also highlighted response time and interface improvements, but their feedback does not establish acceptance in operational work.

During parallel use in the user evaluation, no mixing of conversation information was observed, but this was not a systematic context-isolation test. These observations support further development of a concise interaction, without establishing long-term acceptance.

6.6. Load Tests

Three load profiles repeated the complete-input scenario T01 with the selected configuration. L01 provides the reference load, L02 uses five times as many virtual clients, and L03 applies a higher stress load. L01 and L02 each ran for 10 minutes and 2 seconds, while L03 ran for 48 minutes and 31 seconds. Table 7 reports the mean response times and HTTP error rates from these tests. Absolute client counts and internal capacity requirements are omitted.

At fivefold load, the HTTP error rate remained close to the reference level, while requests took longer on average. The small difference in error rates does not show that higher load improves reliability. Under the stress profile, both response time and the share of failed requests increased substantially. L01 and L02 contained HTTP 500 errors, while L03 also contained HTTP 504 errors. Successful HTTP responses do not establish correct scheduling outcomes.

Table 7: Load-test results for T01. Time is the reported mean response time per request. HTTP error rates count responses with status 500 or 504 among all recorded HTTP calls and are rounded to two decimal places. These are request-level measures, not rates of correct bookings.

Profile	Load	Mean time [s]	HTTP errors [%]
L01	Reference	7.053	0.91
L02	Fivefold	8.105	0.80
L03	Stress	15.455	9.98

These mean request times cannot be compared directly with the median cumulative scenario times in Tables 3 and 4. The tests also used different durations and do not establish a sustainable production load. They show that response time and technical reliability need to be assessed together under concurrent use.

6.7. Implications for Controlled Integration

Bundling operations can improve efficiency, while an additional model stage does not by itself ensure correctness. The practical choice must therefore consider both task outcomes and resource use. Three lessons follow from the case study. First, separate technical execution, tool behaviour and outcome correctness because these can disagree even within the same run. Second, distinguish configuration selection from later case coverage and user feedback, since these stages answer different questions. Third, make empty results and failures unambiguous at the tool boundary and keep confirmation and validation outside the model's discretion. Compared with a fixed REST-based interface, conversational input adds flexibility but also requires explicit state management, validation and unambiguous tool results. No quantitative advantage over that interface was tested.

7. Limitations and Transferability

The findings apply to the implemented workflow and tested configurations, with limits on causal attribution and generalisation.

7.1. Scope of the Comparison

The small set of differentiated scheduling tools limits transfer to larger tool sets, overlapping capabilities or longer dependencies, where separate planning may be useful. The missing backend-without-planner variant prevents a complete comparison of both design choices. Backend orchestration also changes tool granularity, so its effect cannot be attributed to execution location alone.

The selection stage compared three model versions on two standardised scenarios. With five regular repetitions, one run changes a success rate by 20 percentage points. Five successes do not establish a reliable population-level success rate. T01 and T02 supported selection and settings decisions, so their results are not

independent confirmation of the selected configuration's performance. Later functional and robustness cases broadened coverage for Haiku only and cannot establish superiority over the untested alternatives. Prompts were developed using one evaluated model family, potentially favouring it despite shared prompts and tools across models within each variant.

7.2. Technical and Evaluation Limits

Date processing and error handling cover selected expressions and service responses. Response times also depend on external services. Latency medians exclude technically failed runs and use different sample sizes, so faster responses must be interpreted alongside success counts. The load profiles used different test durations and do not establish sustained production capacity.

Direct prompt-injection tests do not cover indirect attacks or establish comprehensive security. Ten repetitions of each fixed input assess consistency but do not provide 80 distinct attack patterns. The functional cases are reported as coverage checks, without repetition counts from which to estimate a success rate. One case required an implementation adjustment, so the later assessment includes development feedback rather than an unchanged-system validation alone. Concurrent use does not verify context isolation. Brief feedback from three operational users cannot establish long-term acceptance, and six developers cannot substitute for operational experience. Reporting aggregate results without prompts and run-level traces limits independent replication. Generalised case descriptions, settings and evaluation criteria allow the study design to be inspected, while internal identifiers, deployment details, access mechanisms and operational capacity figures are excluded.

7.3. Transferability

Explicit tool boundaries, structured state, deterministic validation, clear results and human confirmation provide design principles for other processes. Their suitability and the choice of orchestration require task-specific evaluation.

8. Conclusion and Future Work

This paper presented the design, selection and evaluation of an MCP-based scheduling assistant under human control. The prototype combines flexible language input with structured dialogue state, bounded tools and application-side validation. Explicit confirmation keeps the booking decision with the user.

Haiku with individual tools and no separate planner was selected after five correct outcomes per selection scenario. The subsequent assessment covered seven functional cases, including one that required clearer handling of empty search results. The 80 direct prompt-injection executions produced no observed control violation, but some disrupted the intended workflow. Feedback from three operational users indicated that flexible input was useful while concise responses and low delays remained important. Together, these stages support a reasoned prototype choice and identify development needs, without establishing a universally optimal architecture. The main lesson is to assess outcome correctness separately from technical execution and to treat configuration selection, functional coverage and user acceptance as distinct sources of evidence.

Future work should complete the comparison with backend orchestration without a planner and introduce independent scenarios, larger tool sets and more repetitions.

Further work should also test whether other available LLMs can be used without major changes to the assistant. An initial assessment should keep prompts and tool definitions unchanged and document any model-specific adjustments needed. Models that support the workflow could then be compared on the same independent scenarios using outcome correctness, response time and cost. The comparison should include the functional and robustness checks that were performed only with Haiku in this study. This would show how much effort is needed to switch models and whether the selected configuration remains suitable as more models become available. Longer user studies, sustained load tests and targeted context-isolation and prompt-injection checks are needed before operational use.

References

[Ant26] ANTHROPIC: Claude platform: Pricing, 2026. Haiku 4.5 token prices and regional pricing. Accessed 1 November 2026. URL: https://platform.claude.com/docs/en/about-claude/pricing. 6

[AWV*19] AMERSHI S., WELD D., VORVOREANU M., FOURNEY A., NUSHI B., COLLISSON P., SUH J., IQBAL S., BENNETT P. N., INKPEN K., TEEVAN J., KIKIN-GIL R., HORVITZ E.: Guidelines for Human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2019), CHI '19, Association for Computing Machinery, pp. 1–13. doi:10.1145/3290605.3300233. 2

[Mod25] MODEL CONTEXT PROTOCOL: Model Context Protocol: Architecture, 2025. Specification revision 2025-06-18. Accessed 28 October 2026. URL: https://modelcontextprotocol.io/specification/2025-06-18/architecture. 2

[QLY*24] QIN Y., LIANG S., YE Y., ZHU K., YAN L., LU Y., LIN Y., CONG X., TANG X., QIAN B., ZHAO S., HONG L., TIAN R., XIE R., ZHOU J., GERSTEIN M., LI D., LIU Z., SUN M.: ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations* (2024). URL: https://openreview.net/forum?id=dHng2O0Jjr. 2

[SDYD*23] SCHICK T., DWIVEDI-YU J., DESSÌ R., RAILEANU R., LOMELI M., HAMBRO E., ZETTLEMOYER L., CANCEDDA N., SCIALOM T.: Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems* (2023), vol. 36. doi:10.52202/075280-2997. 2

[SXZ*23] SONG Y., XIONG W., ZHU D., WU W., QIAN H., SONG M., HUANG H., LI C., WANG K., YAO R., TIAN Y., LI S.: RestGPT: Connecting large language models with real-world RESTful APIs. *arXiv preprint arXiv:2306.06624* (2023). doi:10.48550/arXiv.2306.06624. 2, 6

[WTC22] WU T., TERRY M., CAI C. J.: AI Chains: Transparent and controllable Human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2022), CHI '22, Association for Computing Machinery. doi:10.1145/3491102.3517582. 2